

Module 7

I²C — Protocol + RTL + Basic Testbench

Learning objectives

- RTL block architecture: ``clk_div`` + ``i2c_master`` FSM (START → addr → data → STOP).
- Baseline verification: Directed writes plus inline bus monitor (sample SDA on rising SCL).
- Protocol vs simplification: Real open-drain/ACK vs this repo's push-pull teaching model.
- Path to Module 8: Same DUT with UVM monitor/scoreboard on address and data phases.

Prerequisites

- Modules 1–6.
- Verilator, Make, C++ compiler. No UVM required for this baseline.

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["I²C Protocol Essentials"]

T2["Hands-on testbench (i2c_base)"]

T1 --> T2

T2 --> N["I²C UVM+SV"]

Understand the I²C protocol (start/stop, addressing, ACK/NACK), translate it to RTL (simple master + timing), and verify with a **basic** (...)

Overview

- Module 7 is the I²C **protocol + RTL + basic testbench** module (UVM comes in Module 8).

How to learn this module

Suggested learning path

- Read the detailed I²C learning guide:
I2C_LEARNING_GUIDE.md — what I²C is, what kind of protocol (s...
- Read the protocols + UVM overview:
LEARNING_GUIDE_PROTOCOLS_AND_UVM.md — Part B § 4.
When to Use Ba...

Design architecture

1. Block hierarchy

- top_i2c_baseline → `clk\_div` +
`i2c\_master`; C++ drives
`clk`/`rst\_n`.

Rtl Architecture

Diagram: Rtl Architecture
(render with mmdc when available)
flowchart TB
CPP["sim_main.cpp"]
DIV["clk_div"]
MST["i2c_master FSM: START → addr → data → STOP"]
CPP --> DIV
DIV --> MST

2. I²C master FSM

- States: IDLE → START → ADDR_BITS
→ DATA_BITS → STOP.
- One teaching write: 7-bit `addr`
+ data byte on `scl`/`sda`
(push-pull model).

Verification Architecture

Diagram: Verification Architecture

(render with mmdc when available)

flowchart LR

STIM["start, addr, data_in"] --> DUT["i2c_master"]

DUT --> BUS["SCL / SDA"]

BUS --> MON["inline bus monitor\nsample SDA @ rising SCL"]

MON --> CHK["compare addr+W, data"]

CHK --> PASS["I2C baseline PASS"]

3. Teaching vs real I²C

- No open-drain, ACK/NACK, or arbitration in baseline — learn sequencing first.

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart LR

STIM["addr + data"] --> MST["I2C_master"]

MST --> MON["inline bus monitor"]

MON --> CHK["compare bytes"]

CHK --> PASS["baseline PASS"]

RTL block diagram (reference)

Rtl Architecture

Diagram: Rtl Architecture

(render with mmdc when available)

flowchart TB

CPP["sim_main.cpp"]

DIV["clk_div"]

MST["i2c_master FSM\nSTART → addr → data → STOP"]

CPP --> DIV

DIV --> MST

Module 7: DUT hierarchy and signal flow.

Verification / testbench diagram (reference)

Verification Architecture

Diagram: Verification Architecture

(render with mmdc when available)

flowchart LR

STIM["start, addr, data_in"] --> DUT["i2c_master"]

DUT --> BUS["SCL / SDA"]

BUS --> MON["inline bus monitor\nsample SDA @ rising SCL"]

MON --> CHK["compare addr+W, data"]

CHK --> PASS["I2C baseline PASS"]

Module 7: stimulus, observation, and checking.

i2c_master FSM

```
// Simple I2C master (baseline write-only, single target).
// Teaching example: we treat SCL/SDA as simple push-pull signals, not
full
// open-drain with multi-master arbitration. Focus is on bit timing and
// start/address/data/stop sequencing.
//
// One transaction:
//   - START
//   - 7-bit address + write bit (0)
//   - 8-bit data byte
//   - STOP

`timescale 1ns/1ps

module i2c_master (
    input wire      clk,
    input wire      rst_n,           // Active-low async reset
```

clk_div — SCL timing

```
// Simple clock divider for I2C SCL timing (teaching example).  
// Generates a 1-cycle-wide clk_div_tick pulse every DIVIDER cycles of  
clk.
```

```
`timescale 1ns/1ps
```

```
module clk_div #(  
    parameter int DIVIDER = 8  
) (  
    input  wire clk,  
    input  wire rst_n,          // Active-low async reset  
    output reg  clk_div_tick // One-cycle pulse at divided rate  
);  
    int count;  
  
    always @(posedge clk or negedge rst_n) begin  
        if (!rst_n) begin
```

Execution & simulation flow

How the example runs (toolchain)

- Makefile: Verilator compiles RTL + SystemVerilog testbench into a C++ model
- sim_main.cpp: generates clk/rst_n, calls eval() until \$finish
- Directed test (initial block or C++): drive stimulus, wait for DUT flags
- Self-check: compare outputs; print PASS/FAIL (see terminal demo slide)
- Repo path: module7/examples/<example>/ — run make run from that directory

Example directory layout

- dut/: i2c_master.v, clk_div.v
- top_i2c_baseline.sv: Instantiates DUT + directed test + monitor/self-check
- sim_main.cpp: Clock/reset harness; simulation runs until \$finish

Makefile — i2c_baseline run

```
# Makefile for Module 7 I2C baseline (basic testbench, no UVM)
#
# Usage:
#   make run
#

VERILATOR ?= verilator
TOPLEVEL ?= top_i2c_baseline
DUT_SRC = dut/*.v
RTL_SRC = top_i2c_baseline.sv $(DUT_SRC)
HARNESS = sim_main.cpp

VERILATOR_FLAGS = -sv --timing -Wall -Wno-fatal -Wno-TIMESCALEMOD -Wno-
WIDTHTRUNC

.PHONY: run compile clean
```

Key files to study

Open these in the repo

- ``module7/examples/i2c_baseline/dut/i2c_master.v`` — START/addr/data/STOP FSM
- ``module7/examples/i2c_baseline/top_i2c_baseline.sv`` — stimulus + inline bus monitor
- ``docs/I2C_LEARNING_GUIDE.md`` — protocol reference

Verification & testing methods

1. Inline bus monitor

- Top module detects START;
samples SDA on rising SCL;
rebuilds addr+W and data.

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart LR

STIM["addr + data"] --> MST["i2c_master"]

MST --> MON["inline bus monitor"]

MON --> CHK["compare bytes"]

CHK --> PASS["baseline PASS"]

2. Self-check flow

- Compare captured bytes to driven
`addr`/`data\_in`; print `I2C
baseline test PASS`.

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart LR

STIM["addr + data"] --> MST["I2c_master"]

MST --> MON["inline bus monitor"]

MON --> CHK["compare bytes"]

CHK --> PASS["baseline PASS"]

3. Path to Module 8

- Move monitor into I2cMonitor;
scoreboard splits address and
data phases.

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart LR

STIM["addr + data"] --> MST["I2c_master"]

MST --> MON["inline bus monitor"]

MON --> CHK["compare bytes"]

CHK --> PASS["baseline PASS"]

Directed I2C test

```
.data_in      (data_in),  
    .scl      (scl),  
    .sda      (sda),  
    .busy     (busy),  
    .done     (done)  
);
```

```
// Monitor: detect START, then sample 8 address bits and 8 data  
bits.
```

```
initial begin : i2c_monitor  
    int i;  
    captured_addr_w = 8'h00;  
    captured_data    = 8'h00;
```

```
wait (rst_n === 1'b1);
```

```
// START: SDA falls while SCL high
```

Syllabus topics

1. I²C Protocol Essentials

- Bus idle: SCL=1, SDA=1.
- START: SDA goes 1→0 while SCL is 1.
- STOP: SDA goes 0→1 while SCL is 1.
- Address phase: 7-bit address + R/W bit (0=write, 1=read) sent MSB first.
- Data phase: 8-bit data bytes sent MSB first.
- ACK/NACK (concept): receiver pulls SDA low for ACK on the 9th clock; high means NACK.

2. Hands-on testbench (i2c_baseline)

- Stimulus: ``start``, ``addr``, ``data_in``, ``wait(done)``.
- Monitor/self-check: reconstruct address+W and data from SCL/SDA; compare to expected.

Hands-on examples

Module 7 self-check

```
$ ./scripts/module7.sh --help
```

Terminal: module7_check

```

[0:36m=====
[0:36mModule 7: Demo commands (I²C baseline)
[0:36m=====

From repo root:

# 1) Environment sanity:
verilator --version
make --version

# 2) Run the I²C baseline example (basic TB, no UVM):
cd module7/examples/i2c_baseline
make run

[1:33m[INFO] See module7/EXAMPLES.md and module7/examples/i2c_baseline/README.md for more.
```

Example 1: I²C baseline master

- I²C master FSM: START → address+W → data → STOP
- Inline bus monitor sampling SDA on rising SCL
- Comparing reconstructed address and data bytes to stimulus

Demo: I²C baseline master

```
$ cd module7/examples/i2c_baseline && make run
```

Terminal: ex01_i2c_baseline

The run should print:

```
- `[PASS] I2C baseline: write transaction observed correctly`  
- `I2C baseline test PASS`
```

Key concepts

1. Two-wire bus

- SCL + SDA; START/STOP when SCL is high; data changed when SCL is low (simplified model).

2. Address + data phases

- 8 bits address+R/W (MSB first), then 8 bits data — monitor both separately.

Common pitfalls

Ignoring START detection

- Mistake: Sampling bits before a valid START condition.
- Correct: Detect SDA 1→0 while SCL high, then sample on rising SCL.
- Why: Reconstructed bytes align to wrong frame boundary.

Expecting full-spec ACK

- Mistake: Assuming teaching RTL models open-drain ACK/NACK.
- Correct: Treat baseline as sequencing practice; ACK comes in extensions.
- Why: Our push-pull teaching DUT simplifies the bus.

Practice & assessment

Exercises

- Run i2c_baseline
- Trace one transfer
- Bus monitor
- Optional: Waveforms

Assessment checklist

- Can describe I²C START, STOP, address+R/W, and data phase (and ACK/NACK concept).
- Can explain the role of i2c_master and clk_div; know the baseline uses push-pull (simplified).
- Can run i2c_baseline and interpret the test result.
- Ready for Module 8: I²C UVM+SV verification.

Summary & next steps

- Understand the I²C protocol (start/stop, addressing, ACK/NACK), translate it to RTL (simple master...
- Complete module7/CHECKLIST.md
- Review module7/EXAMPLES.md and run each lab
- Next: I²C UVM+SV